

Ansible Blocks and Error Handling:

- In playbooks, blocks are clauses that logically group tasks, and can be used to control how tasks are executed.

EXAMPLE

```
``yaml
```

```
- name: block example
```

```
  hosts: centnodes
```

```
  tasks:
```

```
    - name: installing and configuring Yum versionlock plugin
```

```
      block:
```

```
        - name: package needed by yum
```

```
          yum:
```

```
            name: yum-plugin-versionlock
```

```
            state: present
```

```
        - name: lock version of tzdata
```

```
          lineinfile:
```

```
            dest: /etc/yum/pluginconf.d/versionlock.list
```

```
            line: tzdata-2016j-1
```

```
            state: present
```

```
          when: ansible_os_family == "RedHat"
```

```
...`
```

- Blocks also allow for error handling in combination with the rescue and always statements.
- If any task in a block fails, tasks in its rescue block are executed in order to recover.
- After the tasks in the block clause run, as well as the tasks in the rescue clause if there was a failure, then tasks in the always clause run. To summarize:
- **block:** Defines the main tasks to run.
- **rescue:** Defines the tasks to run if the tasks defined in the block clause fail.
- **always:** Defines the tasks that will always run independently of the success or failure of tasks defined in the block and rescue clauses.

EXAMPLE-2

```
``yaml
```

```
- name: block example
```

```
hosts: all
```

```
tasks:
```

```
- name: installing and configuring Yum versionlock plugin
```

```
block:
```

```
- name: package needed by yum
```

```
yum:
```

```
name: yum-plugin-versionlock
```

```
state: present
```

```
- name: lock version of tzdata
```

```
lineinfile:
```

```
dest: /etc/yum/pluginconf.d/versionlock.list
```

```
line: tzdata-2016j-1
```

```
state: present
```

```
when: ansible_os_family == "RedHat"
```

```
rescue:
```

- name: print error message

debug:

msg: "Failed to install or configure yum-plugin-versionlock"

always:

- name: notify completion

debug:

msg: "Block execution finished (success or failure)"

...

What does this playbook do?

block section contains two tasks

- 1:- name: package needed by yum:
- This installs the YUM versionlock plugin.
- The plugin allows you to lock specific package versions so they don't get upgraded or downgraded during yum update.
- Without it, YUM will always try to pull the latest available version, which can break compatibility in labs or production.
- 2:- name: lock version of tzdata:
- This writes a line into the versionlock list file.
- It tells YUM: "Do not upgrade or downgrade tzdata beyond version 2016j-1."
- The tzdata package contains timezone data. Locking it ensures your system's timezone rules stay consistent, which is critical for applications that rely on time accuracy (databases, logs, cron jobs, etc.).

rescue:

- prints message in case of failure of block section

always:

- prints a message about block execution.

EXAMPLE-3

playbook validates system requirements, installs Apache + SSL, configures the web server, deploys SSL + HTML files, handles errors safely, opens firewall ports, and restarts the service only when needed.

```
``yaml
- name: Control Review Playbook

  hosts: all

  become: yes

  vars:

    min_ram_mb: 2048

  packages:

    - httpd

    - mod_ssl

  services:

    - httpd

  ssl_cert_dir: /etc/httpd/ssl

  web_config_files:

    - { src: ssl.conf, dest: /etc/httpd/conf.d/ssl.conf }

    - { src: index.html, dest: /var/www/html/index.html }

  web_service: httpd

  tasks:

    # Fail Fast Message

    - name: Show Failed System Requirements Message

      fail:

        msg: "The {{ inventory_hostname }} did not meet minimum reqs."

      when: >

        ansible_memtotal_mb < min_ram_mb or

        ansible_distribution != "RedHat"
```

Install all Packages

- name: Ensure required packages are present

yum:

name: "{{ packages }}"

state: latest

Enable and start services

- name: Ensure services are started and enabled

service:

name: "{{ item }}"

state: started

enabled: yes

loop: "{{ services }}"

Block of config tasks

- name: Setting up the SSL cert directory and config files

block:

- name: Create SSL cert directory

file:

path: "{{ ssl_cert_dir }}"

state: directory

- name: Copy Config Files

copy:

src: "{{ item.src }}"

dest: "{{ item.dest }}"

loop: "{{ web_config_files }}"

notify: restart web service

rescue:

- name: Configuration Error Message

debug:

msg: >

One or more of the configuration changes failed,
but the web service is still active.

Configure the firewall

- name: Ensure web server ports are open

firewalld:

service: "{{ item }}"

immediate: true

permanent: true

state: enabled

loop:

- http

- https

handlers:

- name: restart web service

service:

name: "{{ web_service }}"

state: restarted

...

→ ssl.conf & index.html must be available on your control node for this playbook.

control-review/

└─ playbook.yml

└─ ssl.conf

└─ index.html

🔥 Important Ansible Rule

rescue: and always: only run when the **block itself runs**.

If the block is **skipped**, then:

- block → skipped
- rescue → skipped
- always → skipped

If the block FAILS (a task inside block errors)

✓ The block runs

✓ A task inside the block fails

✓ Then Ansible will:

1. Stop executing the remaining tasks inside the block
2. Jump to the **rescue** section:
3. After rescue finishes → run **always** the section